Credit Card Fraud Detection

ML Project Report | Python | XGBoost | Streamlit

Abstract

This project presents an end-to-end machine learning pipeline for detecting fraudulent credit card transactions. Given the extreme class imbalance inherent in fraud datasets (fraud rate ~0.17%), the system combines unsupervised anomaly detection — Isolation Forest and Local Outlier Factor (LOF) — with a supervised XGBoost classifier, achieving robust detection performance measured by ROC-AUC and Average Precision. SMOTE-based oversampling addresses the imbalance during training. An interactive Streamlit dashboard enables real-time single-transaction prediction and bulk CSV batch analysis.

Introduction

Credit card fraud causes billions in losses annually. Automated fraud detection must balance high recall (catching fraud) against precision (avoiding false alarms), a challenge compounded by severe class imbalance — legitimate transactions outnumber fraudulent ones by roughly 578:1 in real-world data.

This project uses the Kaggle ULB Credit Card Fraud dataset (284,807 transactions, 492 fraud cases). Features V1–V28 are PCA-transformed for confidentiality; Amount and Time are the only raw features. The pipeline trains multiple complementary models and serves results through a professional web dashboard.

Tools & Technologies Used

Category	Library / Tool	Purpose
Language	Python 3.8+	Core development language
ML Framework	scikit-learn 1.3	Isolation Forest, LOF, preprocessing
Boosting	XGBoost 2.0	Supervised fraud classifier
Imbalance	imbalanced-learn 0.11	SMOTE / SMOTETomek oversampling
Dashboard	Streamlit 1.28	Interactive web application
Visualisation	Plotly / Matplotlib / Seaborn	Charts & evaluation plots
Data	pandas / numpy	Data wrangling & feature engineering
Persistence	joblib	Model serialisation (.pkl)
Dataset	Kaggle ULB (creditcard.csv)	284,807 real anonymised transactions

Steps Involved in Building the Project

1. Data Acquisition: Downloaded the ULB Credit Card Fraud CSV from Kaggle (via Kaggle API). Dataset contains 284,807 rows and 31 columns.

2. Exploratory Data Analysis: Analysed class distribution, feature distributions (V1–V28, Amount, Time), and correlation heatmaps inside a Jupyter notebook.

3. Preprocessing: Applied StandardScaler to Amount and Time. Performed stratified 80/20 train-test split, then applied SMOTE on the training set to balance classes.

4. Anomaly Detection: Trained Isolation Forest and LOF with configurable contamination rate. Evaluated both via ROC-AUC on the held-out test set.

5. XGBoost Classifier: Trained XGBoost on the SMOTE-balanced training data. Evaluated using confusion matrix, ROC-AUC, Precision-Recall curve, and Average Precision.

6. Model Serialisation: Saved all models (XGBoost, Isolation Forest, LOF, scaler) to the models/ directory using joblib for later inference.

7. Streamlit Dashboard: Built a four-page app: Overview (metrics), Predict Transaction (manual input + gauge chart), Batch Prediction (CSV upload + download), Model Insights (ROC, confusion matrices, feature importance, anomaly score plots).

8. Testing & Validation: Unit tests in tests/test_pipeline.py verify data loading, preprocessing, model output shapes, and prediction interface contracts.

Conclusion

The project successfully demonstrates a production-ready fraud detection pipeline. By ensembling unsupervised anomaly detectors with a supervised XGBoost classifier, and by applying SMOTE to counter class imbalance, the system achieves strong ROC-AUC performance on an inherently difficult dataset. The Streamlit dashboard makes the models accessible to non-technical stakeholders, supporting both real-time and batch fraud screening workflows.

Future enhancements could include online/streaming model updates, SHAP-based explainability per prediction, deployment to a cloud platform (e.g. Streamlit Community Cloud or AWS), and integration of a live transaction data feed via API.